

Machine Learning Basics

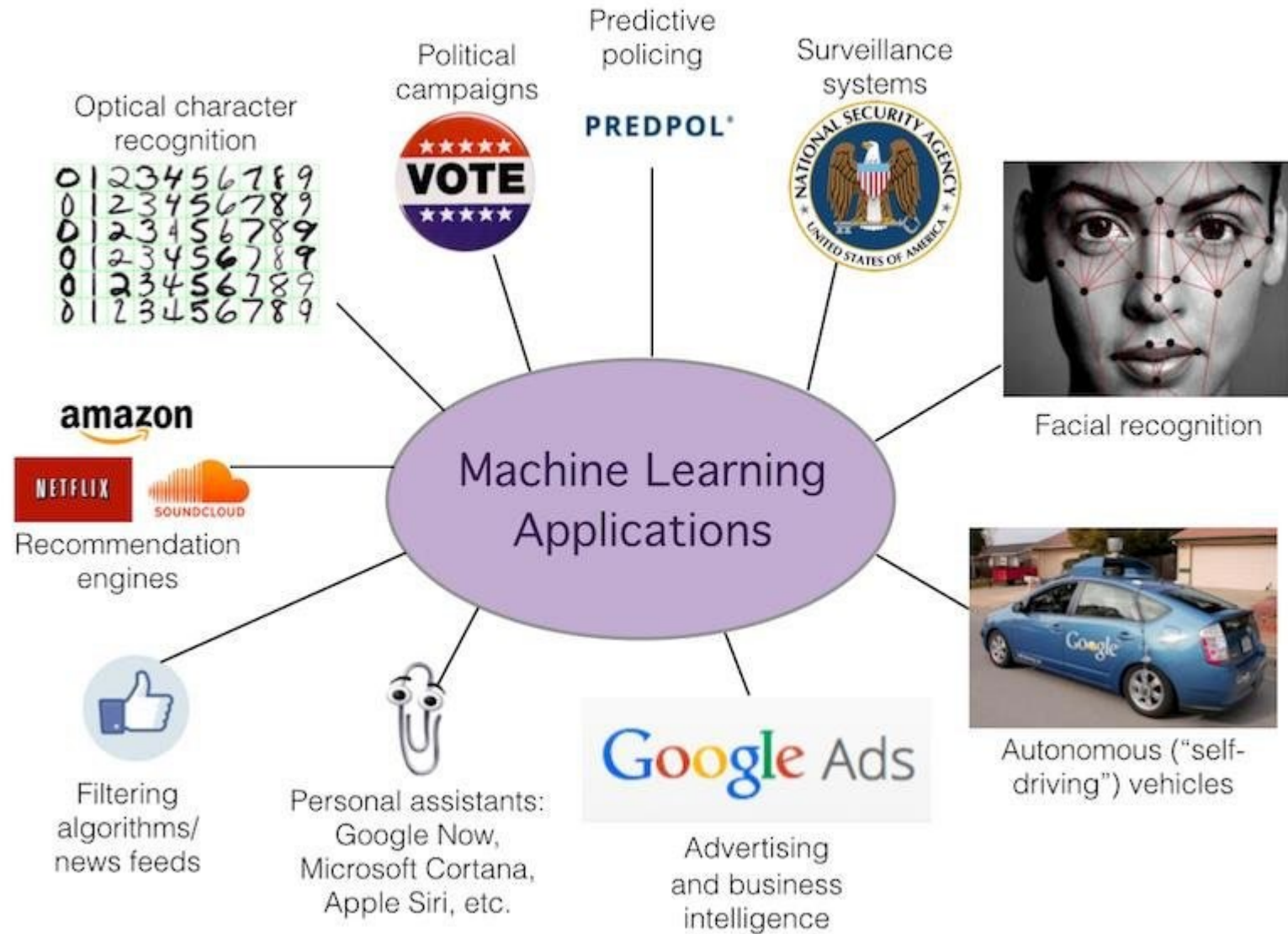
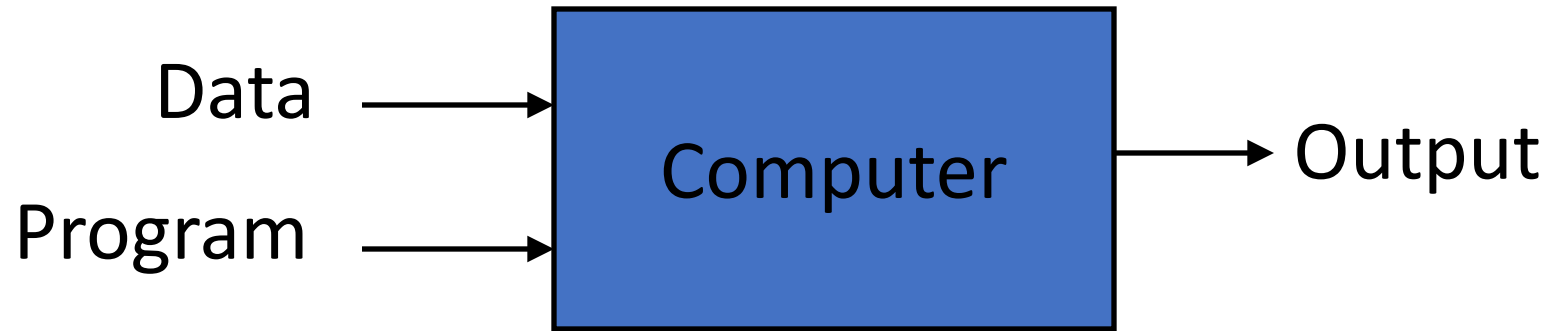
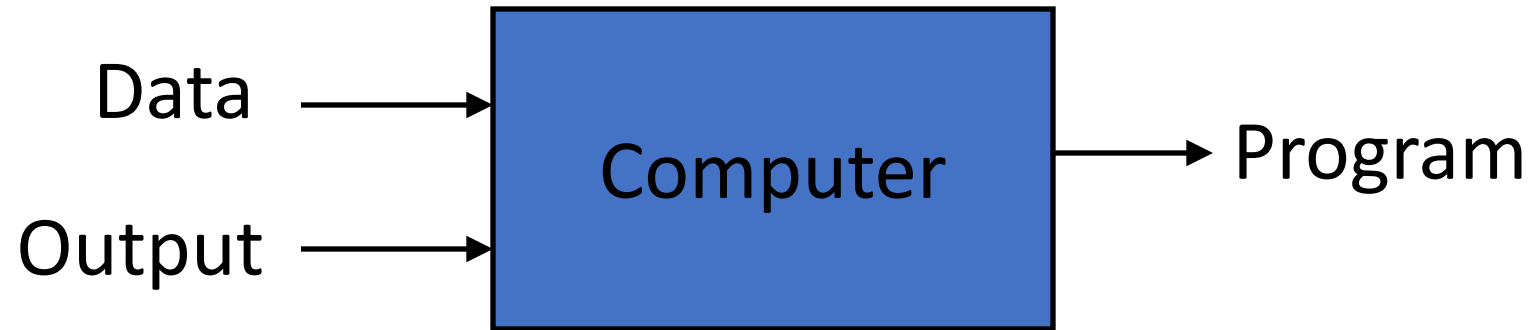


Figure from Ahmad F. Al Musawi

Traditional Programming



Machine Learning



Types of Learning Task

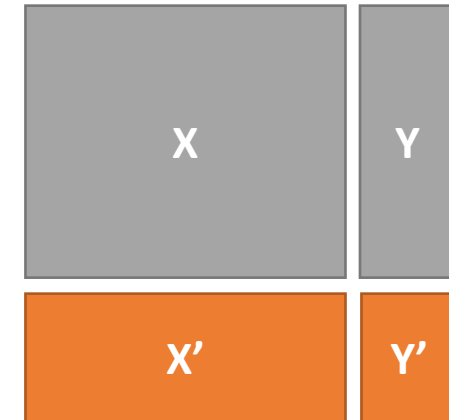
- **Supervised learning**
- **Unsupervised learning**
- **Semi-supervised learning**
- **Reinforcement learning**
- **Self-supervised learning**
- **Meta learning**

Supervised Learning

- **Given** examples of a function $(X, Y = f(X))$
- **Estimate** function $f(X)$ to predict Y for new examples X
 - Discrete Y : Classification
 - Continuous Y : Regression

Supervised learning

- Would like to do **prediction**:
estimate a function $f(x)$ so that $y = f(x)$
- Where y can be:
 - **Real number**: Regression
 - **Categorical**: Classification
- Data is **labeled**:
 - Have many pairs $\{(x, y)\}$
 - x ... vector of binary, categorical, real valued features
 - y ... class: $\{+1, -1\}$, or a real number



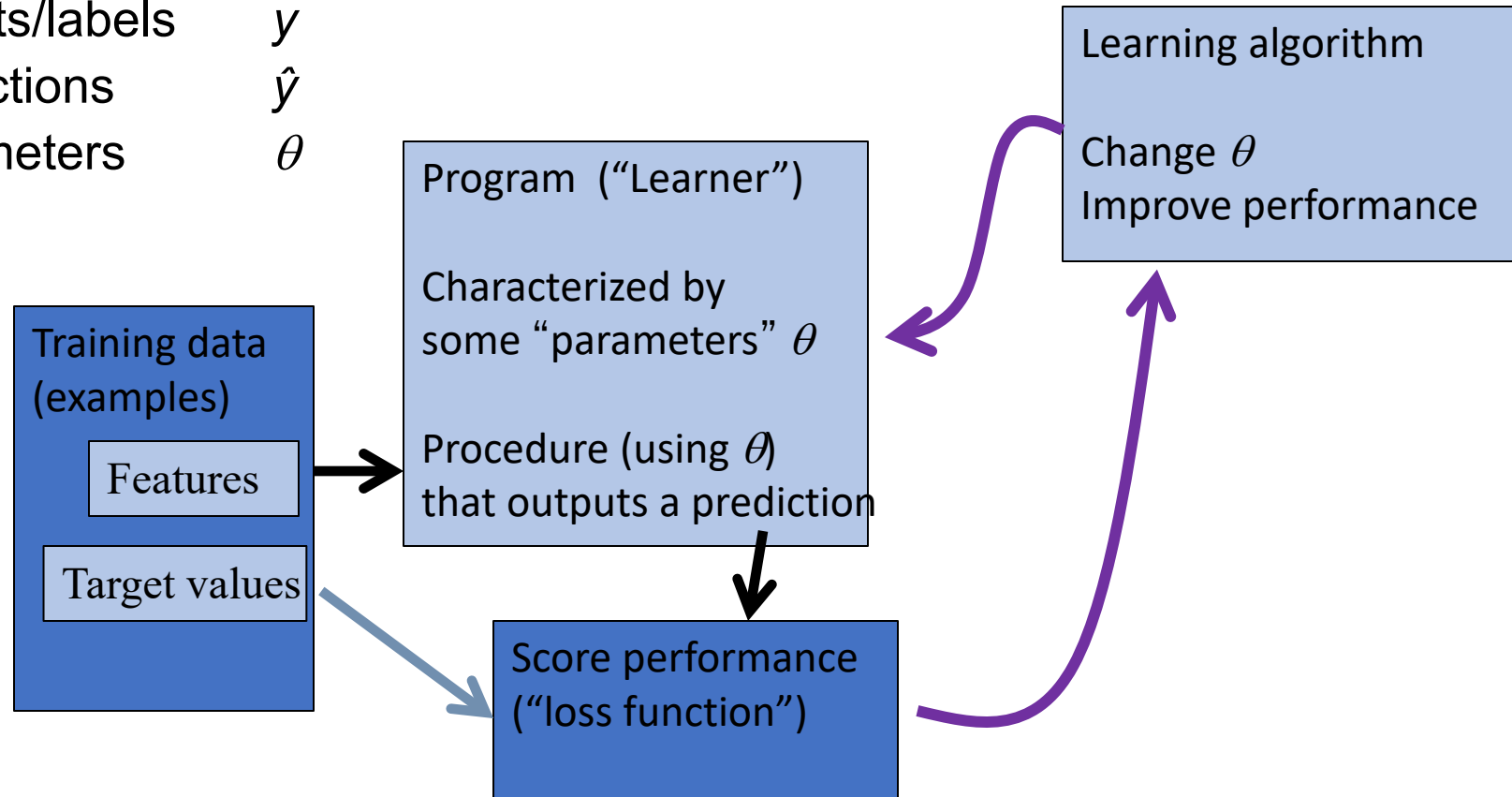
Training and **test** set

Estimate $y = f(x)$ on X, Y .
Hope that the same $f(x)$
also works on unseen X', Y'

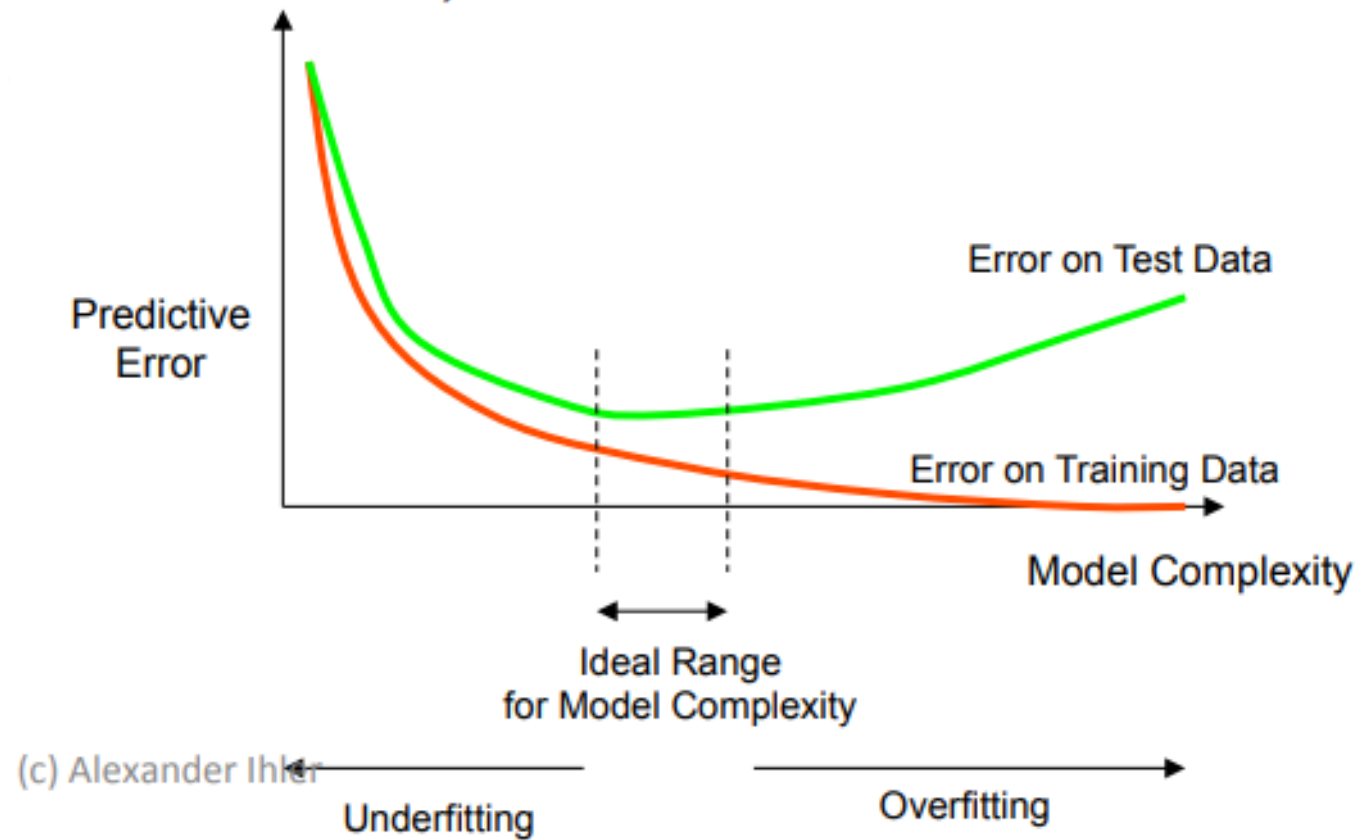
Supervised learning - Training

- Notation

- Features x
- Targets/labels y
- Predictions $\hat{y}$
- Parameters θ



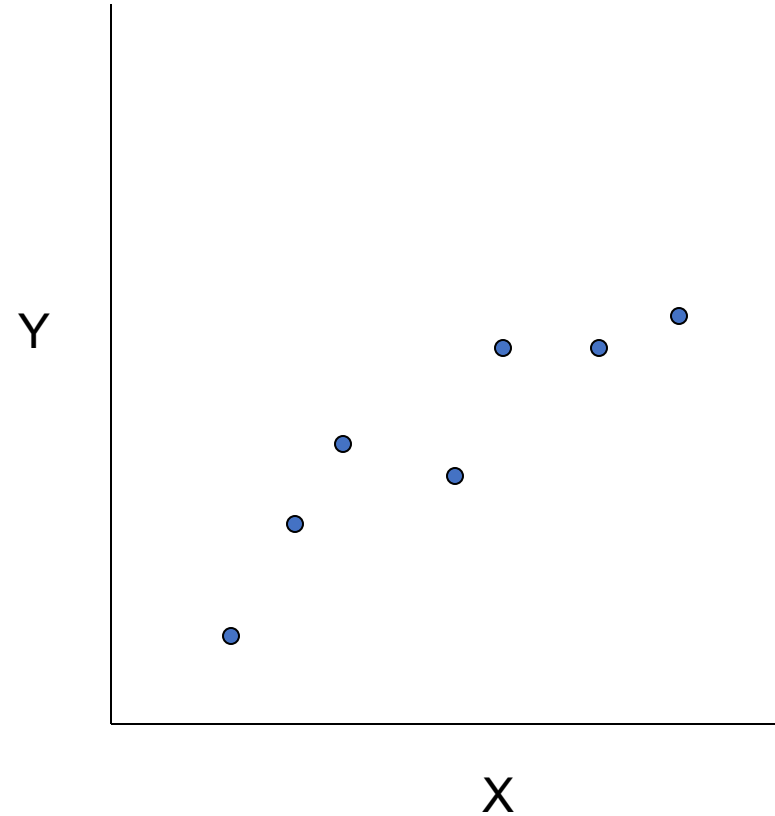
Underfitting and overfitting



Linear Regression

Linear regression

- Given an input x we would like to compute an output y
- For example:
 - Predict height from age
 - Predict Google's price from Yahoo's price
 - Predict distance from wall from sensors



Linear regression

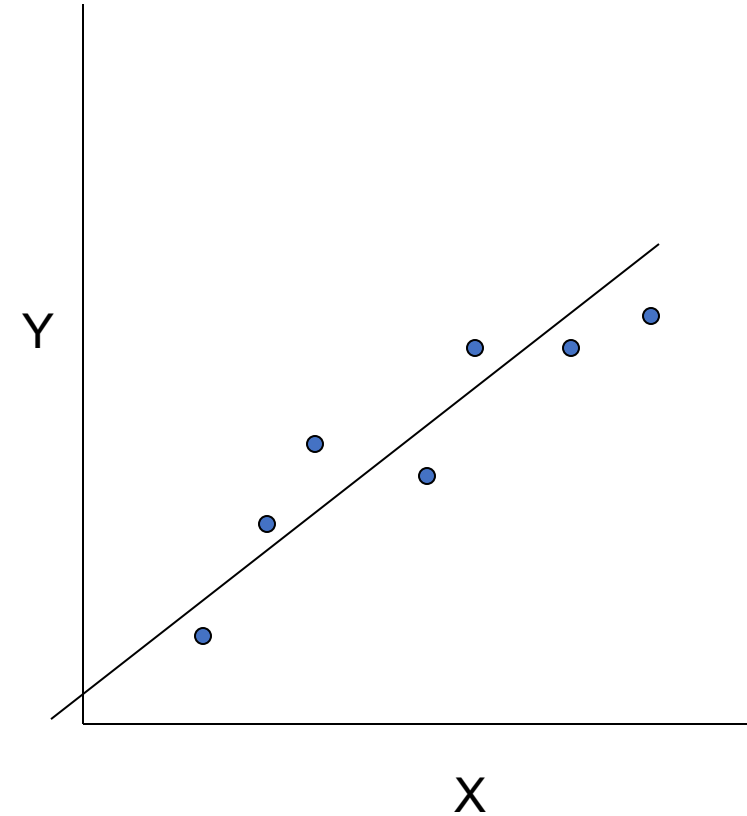
- Given an input x we would like to compute an output y
- In linear regression we assume that y and x are related with the following equation:

What we are
trying to predict

$$\hat{y} = f(x) = w_0 + w_1 x$$

Observed values

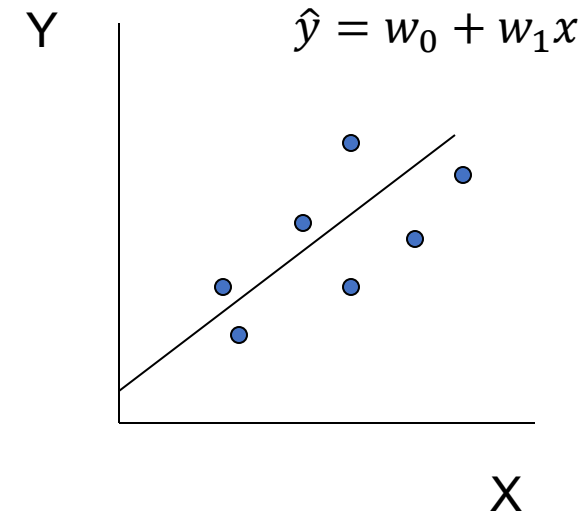
where w_0, w_1 are parameters



Linear regression

- Our goal is to estimate w_0, w_1 from a training data of $\langle x^{(i)}, y^{(i)} \rangle$ pairs
- Optimization goal: minimize squared error (least squares):

$$\operatorname{argmin}_{w_0, w_1} \sum_i (y^{(i)} - w_0 - w_1 x^{(i)})^2$$



Linear Regression: Model

- $f(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$

n features

- Define “feature” $x_0 = 1$, then

- $f_{\theta}(x) = \theta^T x = \theta \cdot x = \theta x$

$$\theta = \begin{pmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_n \end{pmatrix} \quad x = \begin{pmatrix} 1 \\ x_1 \\ \vdots \\ x_n \end{pmatrix}$$

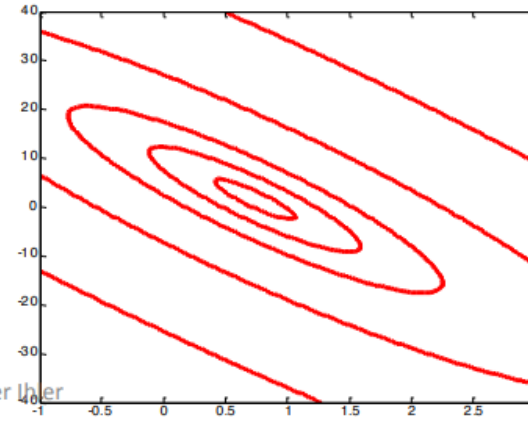
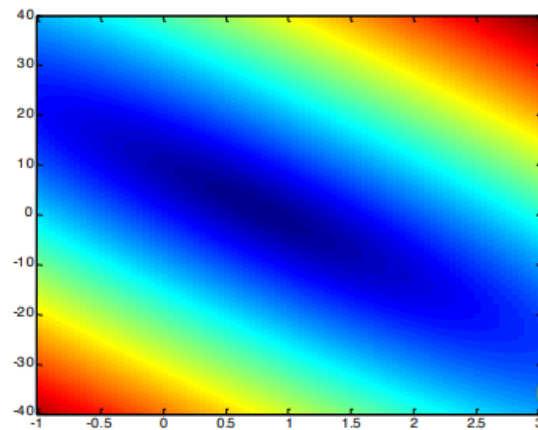
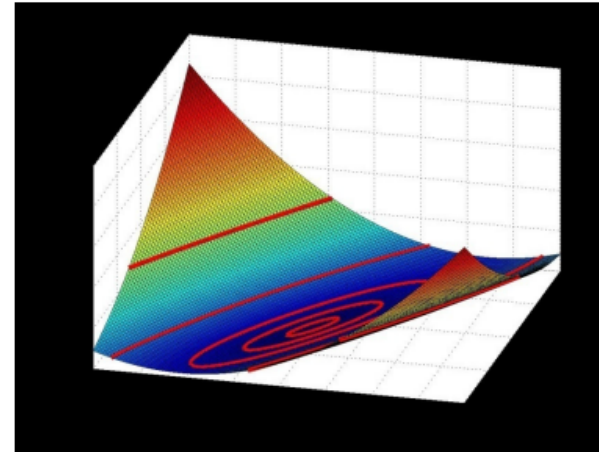
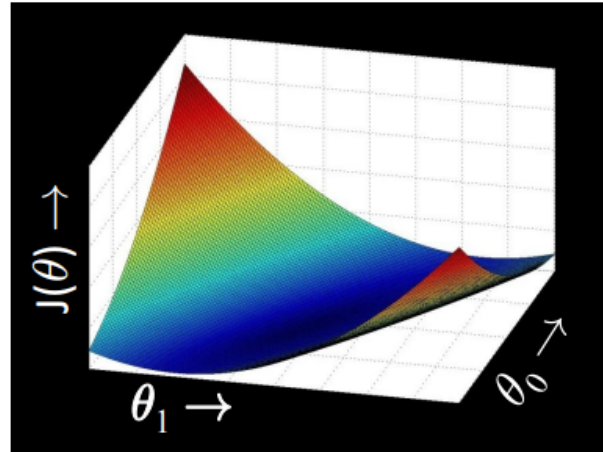
Linear Regression: Loss Function

- m examples in the data: $(\mathbf{x}^{(1)}, y^{(1)}), (\mathbf{x}^{(2)}, y^{(2)}), \dots, (\mathbf{x}^{(m)}, y^{(m)})$
- Loss function (squared error)

$$J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m \left(y^{(i)} - f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) \right)^2$$

$$= \frac{1}{m} \sum_{i=1}^m \left(y^{(i)} - \boldsymbol{\theta}^T \mathbf{x}^{(i)} \right)^2$$

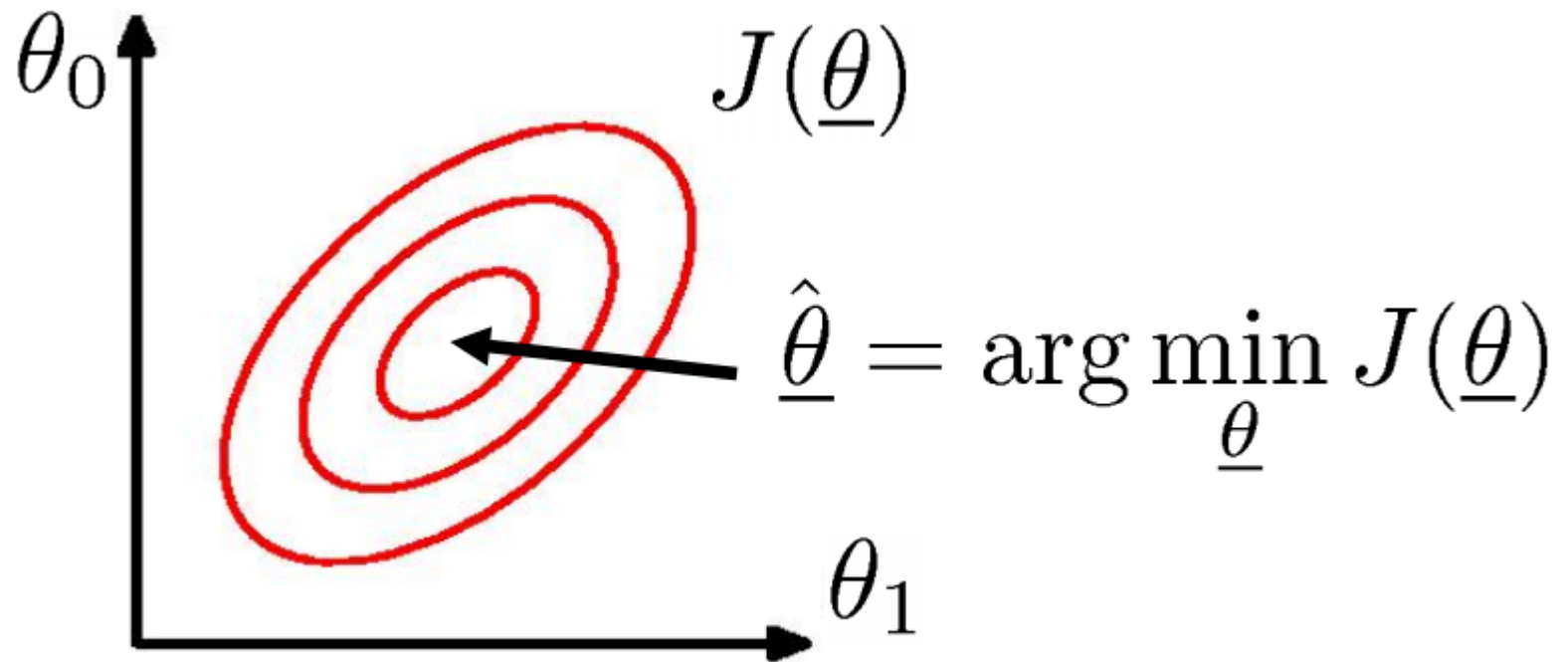
Linear Regression: Optimization



(c) Alexander Ihler

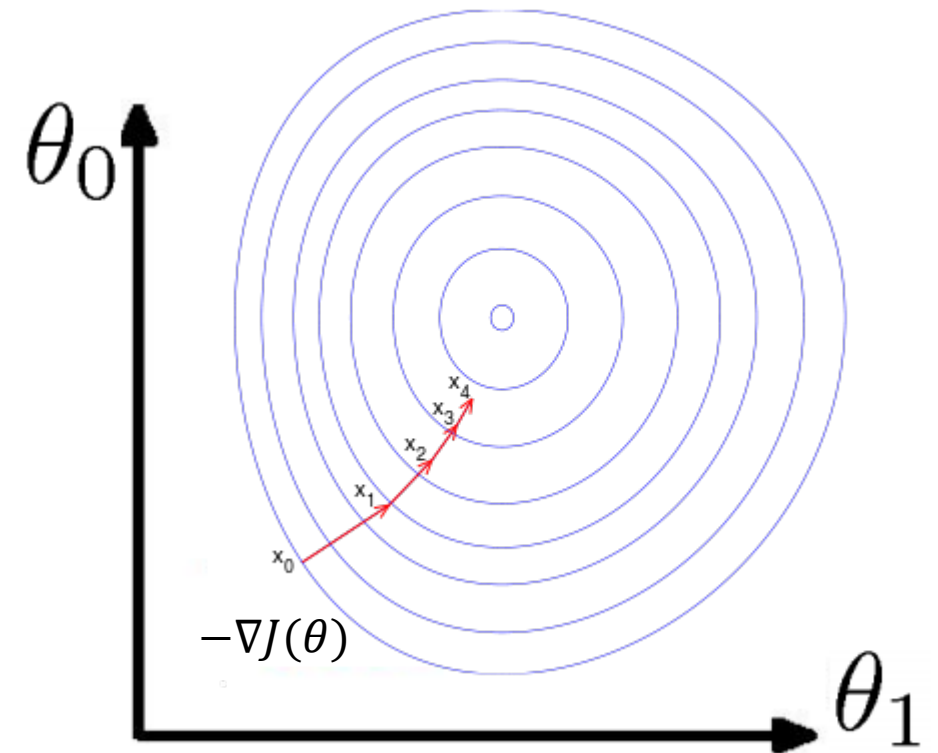
Gradient descent

- Want to find parameters which minimize the loss



Gradient descent in more dimensions

- Gradient vector
- $\nabla J(\theta) = \left(\frac{\partial J(\theta)}{\partial \theta_0}, \frac{\partial J(\theta)}{\partial \theta_1}, \dots \right)$
- Indicates direction of steepest ascent
- (negative = steepest descent)



Gradient descent

- Initialization
- Step size
 - Can change as a function of iteration
- Gradient direction
- Stop condition

Initialize θ

Do {

$$\theta \leftarrow \theta - \alpha \nabla J(\theta)$$

} while (stop condition)